

Long Form Project - Project Plan Milestone 1

Course Code: DIGA3002A

Assignment: LFP Milestone 1

Group Members:

Gemma Ferguson (2653435)

Wilrich van Deventer (2801594)

Phemelo Tsogang (2543064)

Group Number: 5

* Red stars indicate work that has been newly added or work that has been changed for this submission.

Table of Contents

*** Scope**

- Deliverables
- Scope Boundaries
- Scope Creep Solution

Project Management

- Overview
- Github Rules
- Github Issues
- Coding Practices

Project Dependencies

- Phemelo
- Gemma
- Wilrich

*** Project Timeline**

Risk Management

Resources

- Software

*** Handling and Evolution**

- Handling of Workload
- Project Evolution

*** Future Planning**

External Links

Scope

Deliverables

- Our game will include 4 Levels.
 - **Level 1:** Serves as a tutorial to learn the game mechanics and movement system. No narrative involvement from the main antagonist (The Virus). * *There will be narrative involvement from the main protagonist (you, the project developer) to set narrative tone and provide context.*
 - **Level 2:** Introduces The Virus through dialogue. Gameplay mechanics will be similar to level 1, but feature an increase in difficulty.
 - **Level 3:** A level that challenges the player by testing what they learnt in Level 1 and Level 2. The Virus has increased dialogue and begins to try to push and guide the player.
 - **Level 4:** The final level of play. This level will complete the narrative story and feature new mechanics. These mechanics will be introduced by The Virus to try to upend player expectations of the mechanics learnt in level 1-3.
- Total of 8 Maps.
 - *Definition of a Map: this a tile, grid-based layout where gameplay will occur. This will feature a start and end point alongside obstacles/puzzles for the player to overcome.*
 - **2 maps per level** (one normal/corrupted and one 'safe'), 1 map per Mode.
 - **Normal/Corrupted Map:** This is the starting map of all levels and has a start and end point. Obstacles will block the player from reaching the end point, forcing players to strategically switch to the safe map. The safe map and normal map will have different layouts. The normal map will become a Corrupted Map as The Virus takes over, constantly changing map layout and player mechanics.
 - **Safe Map:** This is the alternative map that players can access during a level. This map is different to the Normal/Corrupted map as it doesn't have an end point.
- Player Mechanics
 - **Input System:** A system that reads the player's input which can turn on / off specific player input.
 - **Movement System:** A system that moves the player on the grid. Players can move using WASD or arrow keys. Players can only move in cardinal directions.
 - **Inventory System:** A system that allows players to access an item storage area (inventory). * *This can be seen on the top of the screen at all times.* Players can move items within the inventory as well as drop items back onto the grid.
 - **Interaction System:** A system where the player can pick up items and put items in puzzle slots. The system also allows players to open doors and interact with different elements/obstacles around the map (e.g. pushing a block from one grid space to another or pressing buttons).

- Map Switch Mechanic
 - A mechanic which allows players to switch from one map to another during play. Switching will occur at players' own discretion.
 - Switching of maps includes: Change in visual assets of the map, change in map layout, removal of the effects of The Virus, limiting/unrestricting the player's mechanics.
- Puzzles
 - **Lock and *Unlock Puzzle:** The player will be presented with a lock. They then need to traverse the map to find a code * or a key, which will need to be correctly put into the lock. If done, the lock will remove itself, completing the puzzle.
 - **Item Location Puzzle:** The puzzle will describe a specific item that the player must find and place on a specified location to complete the puzzle.
- The narrative gameplay
 - Our game will have an **overarching narrative that is explored through gameplay and dialogue**. This dialogue will be between you, the player, and the antagonist, The Virus.
 - **The storyline** is that you are a game developer who is playtesting a game you have created. You are progressing through the game's levels when you encounter a virus in your code. This virus is talking to you and observing you as you play the game. Upon realising the power of The Virus as it changes the game you built, you try to delete it. This enrages The Virus and you must try to defeat it and remove it from your game.
- The Virus's gameplay functions
 - **Removal of Mechanics:** In level 4, The Virus is able to remove mechanics from the player that were previously available. This is done to surprise the player and subvert expectations of gameplay.
 - **Adding of Obstacles:** In level 3 and 4, The Virus can 'corrupt' pathways. This will be done through adding obstacles to stop the player's movement.
 - **Setting Traps:** The Virus can set traps in level 4, which will cause the game to end if triggered.
 - **Corrupting Information:** The Virus can corrupt the information the player needs to progress. See Lock and Code Puzzle.
 - **Corrupting Items:** The Virus can corrupt items that the player needs for puzzles. See Item Location Puzzle.
 - **Corrupting the Map:** In level 3 and 4, The Virus can corrupt parts of the map which will change the tile and obstacle layout.

Scope Boundaries

The group is focusing on making a narrative driven puzzle game where the player is a developer testing their game, while the antagonist is a virus that infiltrated the game. The scope is based around this concept thus all features of the game will stay consistent with this narrative theme.

Scope Creep Solution

If anything from the scope needs to change during the development of the long form project then the group will do the following:

1. Decide if the change fits in with the design goals and hypothesis of the project.
2. Evaluate which systems would have a new dependency on the change or which systems would lose a dependency.
3. Evaluate what would happen to the timeline if we were to implement the change, adjusting dates where necessary.
4. Decide who should implement the change.
5. Evaluate the risks of implementing the change.
6. Implement the change.

Project Management

Overview

The group has assigned Phemelo as the project manager. As project manager, this person is responsible for ensuring that development stays on schedule. Alongside this, they must ensure smooth workflow by managing group members and task dependencies. If conflict arises, the project manager is responsible for finding solutions that allow production to continue.

GitHub Rules

Our group has established GitHub rules to try limit risk of corruption and merge conflicts when working together. These are as follows:

1. Work in your own branch. We will merge into the Dev Branch when work is error free.
2. Notify group members before merging your branch into Dev Branch. This can be done in person or via Whatsapp.
3. If work is being done in the same branch, notify the group members via Whatsapp that you are in the branch.
4. The main branch will be used as a backup branch that gets updated at different stages of production.

Github Issues

GitHub Issues will be used as the main system for organising, tracking, and managing tasks throughout the development of the project. Each task / feature will be created as an **Issue**, allowing group members to clearly see what needs to be completed and who is responsible for it.

Issues will be linked to specific Git branches, with members creating a separate branch for the work they are assigned before making changes to the project. Once the work is completed and tested, and working, the branch can be merged and pushed into the main dev branch. This workflow helps prevent multiple members from making conflicting changes to the same files and provides a clear record of what changes were made, why they were made, and who made them.

Issues can also be closed once their associated work has been successfully merged, allowing the team to track the project's progress and maintain an organised development history.

Coding Practices

Established coding practices keep workflow clear and create cohesion in scripts. This allows for group members to work on other members' scripts and improve group efficiency. Some of these established coding practices to follow are:

- Name variables and scripts after what they do in-game. See the example below:

```
//Private variables.  
private float _variableName;  
//Public variables.  
public float VariableName;  
//Constants.  
const float VARIABLENAME = 0F;  
  
//Functions should be named after what they do (MovePlayer,PickUpItem).  
private void FunctionName(float variableName)  
{  
    //You don't need to add "function" before the variable name.  
    float functionVariableName = 0f;  
}
```

Figure 1: Example of naming conventions to be followed in scripts.

- Make use of modular coding so that code functions can be called by other scripts.

```
private void Start()
```

```

{
    for(int i = 1 ; i<11; i++)
    {
        Debug.Log(i);
    }
}

```

Figure 2: An example of how NOT to code in the project. Here functions are specific to this script and are not easily called.

```

private void Start()
{
    CountToTen();
}
private void CountToTen()
{
    for(int i = 1 ; i<11; i++)
    {
        Debug.Log(i);
    }
}

```

Figure 3: An example of the desired modular coding. Here functions are named and can easily be called by different scripts.

- When using variables in scripts, avoid making use of fixed or “magic” numbers that have no context outside of the script. By not using magic numbers, it allows for scripts to be easily understood by those who didn’t write them. It also allows for reusability as you generalize functions.

```

private void MoveToPosition()
{
    transform.position = new Vector2(10f,10f);
}

```

Figure 4: An example of coding that uses magic numbers. This is not desired.

```

private void RandomFunction()
{
    Vector2 newPosition = new Vector2(10f,10f);
    MoveToPosition(transform,newPosition);
}

```

```

}

private void MoveToPosition(Transform moveTransform, Vector2 newPosition)
{
    moveTransform.position = newPosition;
}

```

Figure 5: An example of coding that does not use magic numbers. This is the preferred method of writing code.

Project Dependencies

The following table outlines the allocation of work to each group member. This has been done in a step-by-step method from creation to completion. To understand dependencies, go to the dependent task number and find the name of the group member responsible for that task.

No.	Task	Group Member Responsible	Dependencies
<i>Level 1</i>			
1	Creation of base grid-layout for 'safe' map and 'normal' map.	Phemelo/Gemma	None.
2	Creation of obstacles, start and end locations on grid tiles.	Phemelo	Task 1.
3	Create a mechanic that switches the player between the safe and corrupted map.	Wilrich	Task 1.
4	Create the player and player controller to move between tiles on the grid.	Wilrich	Task 1 and 2.
<i>Level 2</i>			
5	Create new tile maps for safe and corrupted map.	Phemelo/Gemma	Task 1.
6	Use the obstacles from Level 1 to create a new level design for level 2.	Phemelo/Gemma	Task 2.
7	Introduce new mechanic to the map which increases difficulty of level.	Phemelo	Task 5.

8	Write and present text as a UI variable that will act as 'The Virus'.	Gemma	The writing can be done without dependencies, but execution with gameplay requires task 5, 6, 7.
<i>Level 3</i>			
9	Create new tile maps for safe and corrupted map.	Phemelo/Gemma	Task 1.
10	Use the obstacles from Level 1 to create a new level design for level 3.	Phemelo	Task 2.
11	Write more of The Virus's text and let its personality become more present.	Gemma	Task 8.
12	Introduce a mechanic that functions as The Virus changing the corrupted map during play.	Wilrich	Task 11.
<i>Level 4</i>			
13	Create new tile maps for the safe and corrupted level.	Phemelo	Task 1.
14	Reinvent how obstacles and elements on the corrupted level function.	Phemelo	Task 13.
15	Keep using the virus mechanics introduced in level 3, but increase their difficulty.	Wilrich	Task 12.
16	Use writing to show the Virus having fully taken over the map and in full control.	Gemma	Task 11.
17	Introduce a mechanic that allows the player to 'destroy' the virus and 'win' the game.	Wilrich	Task 13 - 16.
<i>Polish</i>			
18	Create the main menu screen.	Gemma	None.
19	Create a pause screen.	Gemma	Any level 1 - 4.
20	Implement all finalized assets and sprites into the game.	Gemma/Wilrich	Task 1 - 17.

21	Playtest.	All	Task 1 - 17.
----	-----------	-----	--------------

Task	1 - Main Responsibility	2 - Discussion Pal, Idea Soundboard and Helper	3 - Last Ditch Resort
Virus	Wilrich	Phemelo	Gemma
Player Mechanics	Wilrich	Phemelo	Gemma
Map Switch Mechanic	Wilrich	Phemelo	Gemma
Narrative	Gemma	Wilrich	Phemelo
Level Design	Gemma	Phemelo	Gemma
Puzzles	Phemelo	Gemma	Wilrich
Maps	Phemelo	Gemma	Wilrich
Assets	Gemma	Wilrich	Phemelo
UI	Gemma	Wilrich	Phemelo

Phemelo (Project Manager, Puzzle and Level Designer/Programmer)

Phemelo will be the project manager through the entire development process. This will entail taking a high level look at the project at all times and ensuring that the game is maintaining the core idea/hypothesis proposed in the game design document. This role will also be to ensure that project deadlines and milestones are achieved on time and with the highest quality possible.

In terms of actual game development, Phemelo will be responsible for implementing the grid/tile based game map through which the player, virus and puzzles interact. Using this map, Gemma and Phemelo will cooperate on designing levels which align with the narrative direction and puzzle /player mechanics.

Phemelo will primarily be designing the puzzles which the player must use the map switch mechanics to solve. This involves making the map puzzles progressively more difficult and more demanding on the player's use and understanding of the map switch mechanic and the antagonistic virus which keeps subverting the player's understanding of how to solve that puzzle.

Gemma (Level, UI & Assets Designer and Narrative Writer)

Gemma will primarily focus on writing a narrative that fits in with the core themes of the project. This includes the dialogue and narrative beats of the Virus which provides obstacles for the player. This virus will be providing the context of every decision and solution to game puzzles and thus will need to be contemplated and implemented for and throughout the entire game. This narrative design role will also extend to the level design to ensure that each level progresses the narrative and difficulty.

The Normal mode map will use a flat art style while the Safe mode will have a pixelated art style. This will need 2 different styles for each asset and thus assets will have to be implemented and developed throughout the entire development cycle and not just at the end during the polishing phase. Gemma will primarily undertake the role of creating the assets while any extra assets which exceed the allowable workload will be created by Phemelo and Wilrich. The development of communicative UI and polished UI will also be done by Gemma. This will include how the Virus is conveyed and how the map switch is communicated to the player.

Wilrich (Player Controller, Virus/Enemy Controller and Map Switch Mechanic Programmer)

Wilrich will be the primary developer/programmer for player mechanics, virus mechanics and the designing the map switching mechanics.

Player mechanics development spans from the grid based movement of the player to their interaction with the virus and puzzle gameplay.

The virus will have both a narrative context as an antagonistic force against the coder/player, as well as a mechanical context as an obstacle to the player's puzzle solution by actively adding challenges in the player's way as well as taking away certain mechanics from the player. Wilrich will henceforth be developing the mechanics of the virus while using Gemma's narrative context

Wilrich will also be developing the map switching mechanic. This will take heavy inspiration from the time travel mechanic in The Silent Age. Wilrich will have to develop the cooldown system for this mechanic, manage the disabling and enabling of both maps when the player switches between the two, as well as how seamless the transition between Normal mode and Safe mode state are.

Project Timeline

Empty days are used for bug fixes and breaks.

Working on documentation throughout

Date	Objective
27 August	- Finish Project Plan.
30 August	- "Finish" Game Design Document (To be continuously updated).
31 August	- Finish the slides for presentation.
1 September	Submit LFP Pitch Presentation.
6 September	- Begin work on level 1 of the game. - Have the basic tile system completed and begin level design. - Completed player movement controller.
7 September	- Completed mechanical switching between safe and normal/corrupted map versions. - Start with puzzles found on levels.
9 September	- Finalize the level design for level 1.
11 September	- Level 1 Complete. - Begin work on level 2.
13 September	- Start Virus Code - Create the level design for level 2.
15 September	- Begin work on Main Menu screens, pause screens. - Complete and introduce the Virus's text.
17 September	- Completion of level 2 Virus code.
19 September	- Level 2 Complete. * Incomplete
23 September	Submit LFP Milestone 1.
25 September	- * Finalize level design for level 2.
27 September	- * Start work on new obstacle mechanics for level 3 - * Start level 3 - Begin work on the Virus's corruption mechanics
29 September	- Finish corruption mechanics - Level 3 complete. - Complete the Virus's text
1 October	- Begin work on creating the assets to be used in final gameplay.

3 October	- Start level design for level 4. - Begin work on new obstacle mechanics.
5 October	- * Begin work on the final win system for the level. (moved from original position)
8 October	- Finish Level 4 - Complete obstacle mechanics for level 4.
14 October	Submit LFP Milestone 2
16 October	- Start implementing all assets - Start finishing the narrative
20 October	- Complete the Virus's text
24 October	- Implement all assets that have been made - Finish work on main menu screens and pause screens.
28 October	- Finish game - Start Playtesting
29 October -12 November	- Playtest - Implement playtesting changes - Fix bugs
13 November	LFP Final Submission.

Risk Management

All members are responsible for problems that arise in the project as this is a group project. We are relying on each other to communicate effectively.

Risks and mitigation were researched at [1][2].

Risk	Risk Mitigation	Member(s) responsible
Scope Creep	Keep a close eye on the timeline to ensure that we have enough days to complete scoped work. If the work is continuously deviating from the timeline and submission work is falling short of expectation, it is time to reassess scope.	Phemelo
Vision Drift (differences in	Have in-person or online meetings once weekly to ensure that all group members	All

opinions and understanding)	are on the same page. This also helps keep communication clear between group members.	
Team Burnout	On the project timeline there are gaps in dates between objectives needing to be completed. These gaps are necessary for the team to take a break from the work. The timeline has also been designed incrementally so that members can work at their own pace.	All
Merge Conflicts	Making a backup branch in Github that is periodically (every 2 days) updated after big changes are pushed. Our GitHub rules are additionally in place to try to minimize any errors.	All
Group member cannot work on task	The second or third responsible for the task will work on those tasks	All

Resources

Software

Game Engine: Unity 6.5 (6000.5.9f1)

Version Control: GitHub

Coding Environment: Visual studio code (Add whichever coding environment you want to use)

Task System: GitHub Issues

Communication: Discord, Whatsapp

*** Handling and Evolution**

Handling of Workload

The group has handled the creation of the project well, with work progressing mostly as planned. Between the Pitch Presentation and Milestone 1 we set out to have:

- ☒ Completed tile system for movement.
- ☒ Completed player movement controller.
- ☒ Mechanic of switching between safe and normal mode on the maps.
- ☒ Finalized and completed level 1 and its associated puzzles.

- ☒ Have work done on main menu screens, pause screens and other UI elements.
- ☒ Introduction of The Virus.
- ☐ Complete level 2 and its associated puzzles.

I have ticked the boxes where these objectives were achieved. The only thing we did not achieve was the full completion of level 2. Regardless of this, the level still demonstrates a basic level design and it is playable. The primary element missing is The Virus's influence. This relates to its changing of gameplay elements and narrative voice. The timeline has been adjusted accordingly to reflect the impact this incomplete objective has on overall production. It's a minor impact with our goals and objective for Milestone 2 being largely the same.

Project Evolution

Areas where the project plan has been altered to address changes to the idea of the project can be seen by a red star next to them. These changes have been summarized as follows:

Scope - Slight alterations have been made to the scope. These relate to details such as puzzles now being 'lock and unlock' puzzles instead of 'lock and code' puzzles. This allows for multiple ways to complete the locks seen on the levels. Another scope change is that narrative will now be present on level 1. This narrative will come from you, the game developer, to provide context to the player of the game's storyline.

Project Timeline - The completion of level 2 has been shifted to after the submission of Milestone 1. This means we have slightly more work to complete before Milestone 2 submission, but the foundational aspects of level 2 are in place so work isn't largely delayed.

* Future Planning

The creation of our Alpha Build Prototype has taught us

This will impact our future by changing

External Links

[1] L. Wickham, "9 Critical Game Development Risks and How to Mitigate Them," *Game Production Academy*, Jan. 30, 2026.

<https://www.game-production.com/post/9-critical-game-development-risks-and-what-to-do-about-them> (accessed Aug. 28, 2026).

[2] P. Tozour and 2013, "Managing Risk in Video Game Development," *Game Developer*, May 03, 2013. <https://www.gamedeveloper.com/business/managing-risk-in-video-game-development> (accessed Aug. 28, 2026).